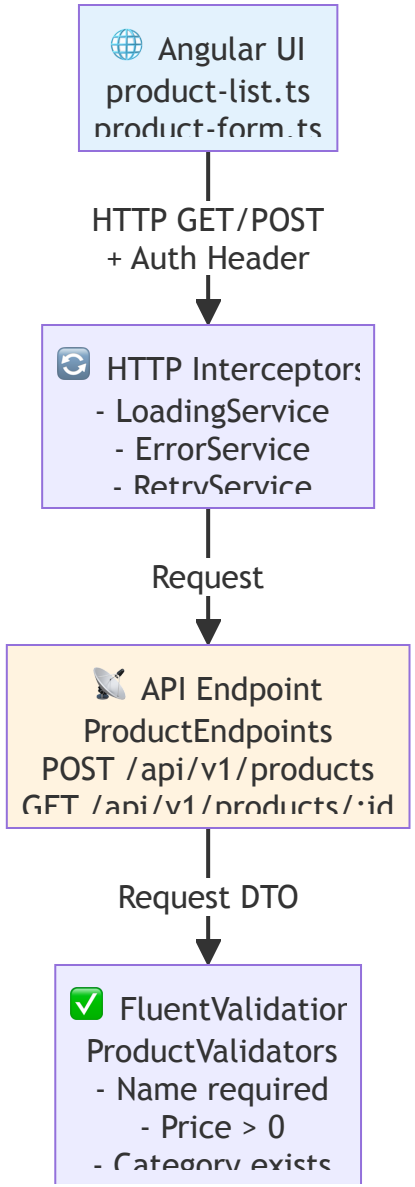
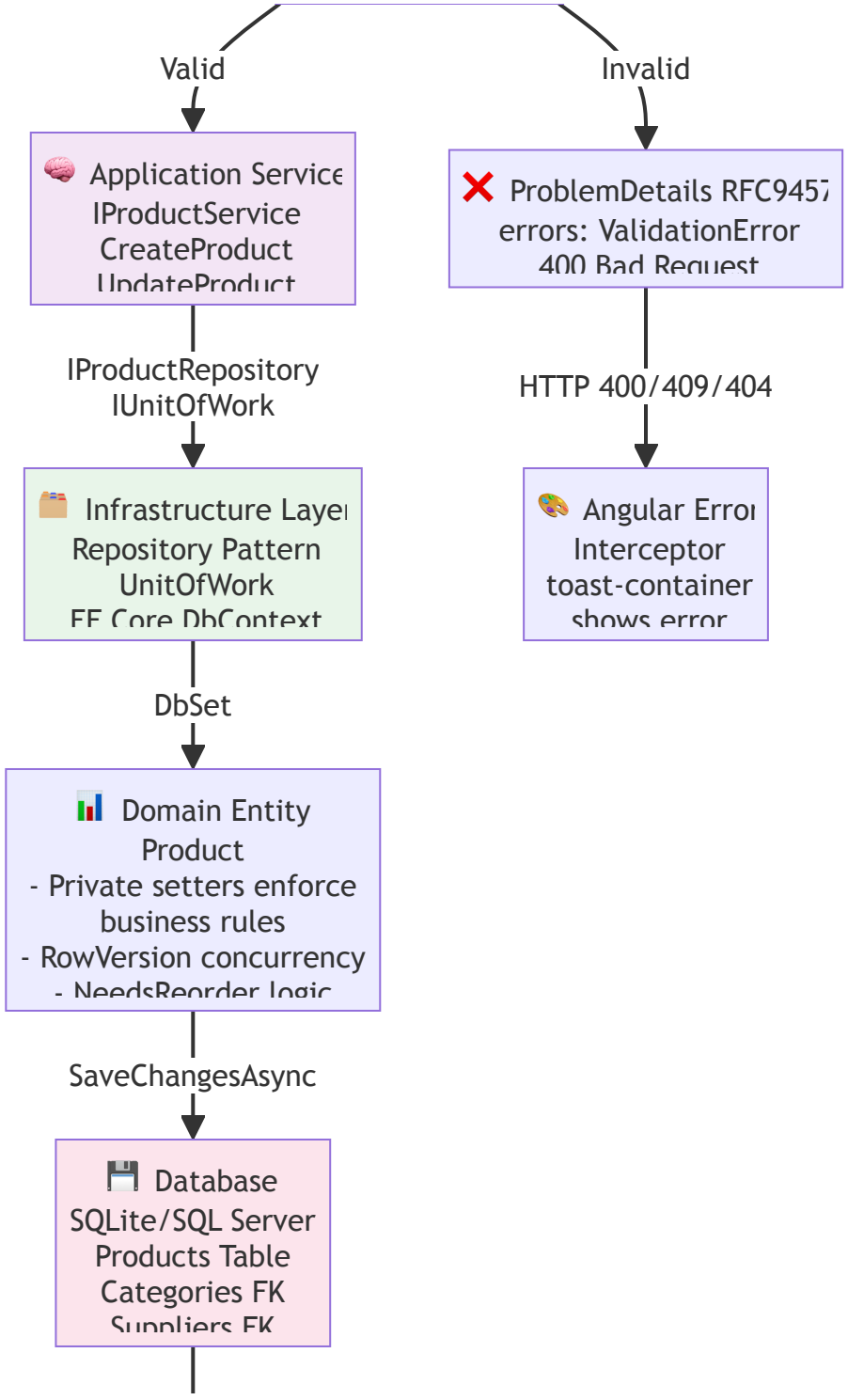
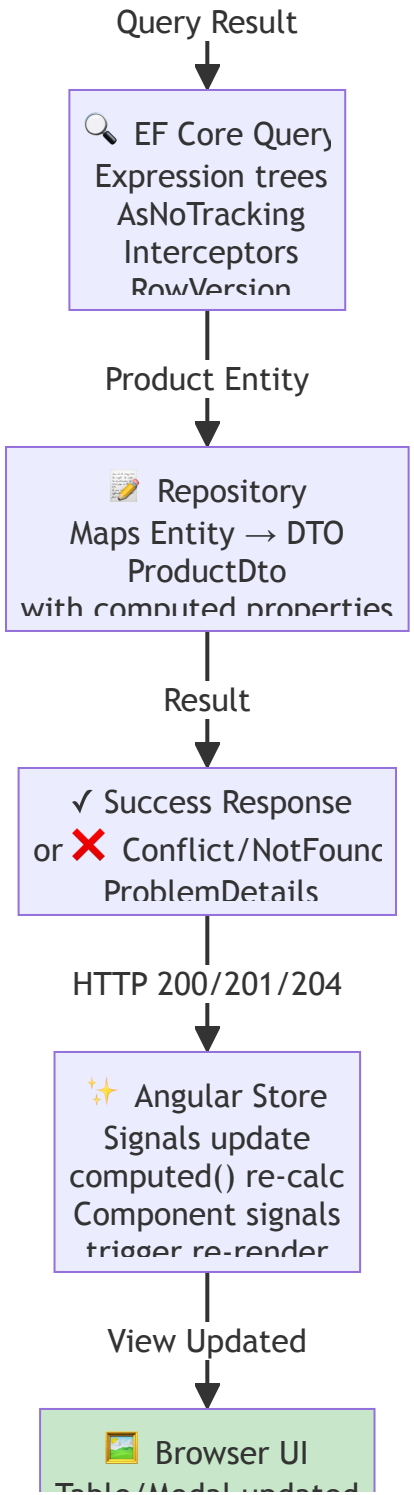
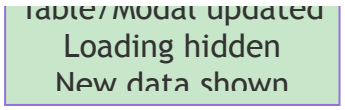
Complete Request/Response Cycle




Valid
Invalid
Application Service
IProductService
CreateProduct
UpdateProduct
ProblemDetails RFC9457
errors: ValidationError
400 Bad Request
IProductRepository
IUnitOfWork
HTTP 400/409/404
Infrastructure Layer
Repository Pattern
UnitOfWork
EF Core DbContext
Angular Error
Interceptor
toast-container
shows error
DbSet
Domain Entity
Product
- Private setters enforce
business rules
- RowVersion concurrency
- NeedsReorder logic
SaveChangesAsync
Database
SQLite/SQL Server
Products Table
Categories FK
Suppliers FK


Query Result
🔍 EF Core Query
Expression trees
AsNoTracking
Interceptors
RowVersion
Product Entity
Repository
Maps Entity → DTO
ProductDto
with computed properties
Result
✓ Success Response
or ❌ Conflict/NotFound
ProblemDetails
HTTP 200/201/204
✨ Angular Store
Signals update
computed() re-calc
Component signals
trigger re-render
View Updated
Browser UI



How Data Flows Through Each Layer

Frontend (Angular)

- **product-api.service.ts** - Makes HTTP calls to API
- **product.store.ts** - Signal Store holds state, uses signals and computed for reactivity
- **http.interceptors.ts** - Adds loading state, handles errors, retries failed requests
- **product-list.ts** - Component subscribes to store, renders data via signals

API Layer (.NET)

- **ProductEndpoints.cs** - Maps HTTP verbs to handlers, binds JSON body → CreateProductRequest
- **ResultExtensions.cs** - Converts Result<T> to TypedResults (Ok, Created, BadRequest, Conflict)
- **GlobalExceptionHandler.cs** - Catches unhandled exceptions, returns ProblemDetails
- **Output Caching** - Caches GET responses by tag, evicts on mutations

Application Layer

- **ProductService.cs** - Orchestrates business logic, calls validators and repository
- **FluentValidation** - Validates request data before touching the database
- **Result<T>** - Railway-oriented programming: Success or Failure with error details
- **ConcurrencyConflictException** - Caught and converted to 409 Conflict response

Domain + Infrastructure

- **Product Entity** - Private setters ensure only domain methods (Create, Update, Apply) modify state
- **ProductRepository** - Implements IProductRepository: Add, Remove, GetByIdAsync, SetExpectedRowVersion
- **ProductQueries** - Implements IProductQueries: Search, filter, sort, pagination via expression trees
- **EF Core DbContext** - Change tracking, optimistic concurrency (RowVersion), interceptors for auditing

Example: Create Product Flow

1. **Angular:** User fills form, clicks Save → POST /api/v1/products with CreateProductRequest (name, supplierId, etc.)
2. **HTTP Interceptor:** Adds auth header, sets loading = true
3. **ProductEndpoints:** Receives CreateProductRequest, deserializes JSON
4. **ProductValidators:** FluentValidation checks name is not empty, price > 0, category exists in DB
5. **ProductService.CreateAsync():** If validation passes → calls repository.Add() and unitOfWork.SaveChangesAsync()
6. **Product.Create():** Domain factory enforces rule "ProductName must not be empty" (via guard clause)
7. **EF Core:** Interceptor adds CreatedAtUtc timestamp, inserts into Products table
8. **Database:** SQL Server/SQLite inserts row, generates RowVersion (concurrency token)
9. **Repository:** Maps Product entity → ProductDto with computed fields (CategoryName from JOIN)
10. **ResultExtensions:** Result<ProductDto>.IsSuccess → TypedResults.Created("/api/v1/products/{id}", productDto)
11. **HTTP Response:** 201 Created with Location header, JSON body with new product
12. **Output Cache:** Invalidates /api/v1/products (the tag) so next GET is fresh
13. **Angular Store:** productStore.add(newProduct) updates signal, computed triggers re-render
14. **UI:** Table refreshes, new row appears, loading spinner hidden, toast confirms "Product created"

Key Architectural Principles in Action

✅ **Dependency Inversion:** ProductService depends on IProductRepository (interface), not concrete repo. Repository is injected at startup.

✅ **Single Responsibility:** Validators only validate, Repositories only query/persist, Services only orchestrate business logic.

✅ **Separation of Concerns:** Domain entities live in Domain layer (independent of EF Core), queries in Infrastructure (EF-specific).

✅ **Railway-Oriented Programming:** Result<T> chains successes and failures without throwing exceptions in normal flow.

✅ **Clean Architecture Layers:** Api → Application → Domain ←Infrastructure. Dependencies flow inward only.